[redacted]

School of Informatics & Cybersecurity - Assessment Submission Coversheet

Student Information

Student Number:	[redacted]
Student Name:	Maël Conejos Crusson

Module Information

Module Title:	Security Architecture
Module Code:	[redacted]

Assessment Information

Assessment Title:	Designing and Implementing a Proof of Concept Cloud Solution based on a client's needs

Declaration

☑ I confirm that I have read and understood the University's Academic Integrity Policy.

[redacted]

☑ I have completed the TU Dublin AI Driving License theory test.

[redacted]

☑ I declare that this assessment submission is my own work.

☑ I confirm that I have read and understood the AI traffic light system used as part of this assessment brief and I have adhered to the level of AI usage allowed for this assessment.

Student signature:	[redacted]
Date:	17/04/2026

[redacted]

[redacted]

Security Architecture Project

WinLocal Giveaways & Amazon Web Services

Maël Conejos Crusson
17/04/2026

Table of Contents

Introduction 3
Task 1: Detailed AWS Architecture Diagram 3
Task 2: Proof of Concept – Implementation & Explanation 4
Networking Design 4
Bastion Host 5
Amazon Machine Image (AMI) 7
Single-AZ Amazon RDS Deployment 8
Auto Scaling Configuration 9
Load Balancer Configuration 11
Security Implementation 12
Static Backup Website 13
Task 3: Justification of Additional AWS Services 14
Route 53 14
Web Application Firewall 15
Task 4: Sustainability – AWS Well-Architected Framework 15
Conclusion 16
References 16

Introduction

WinLocal Giveaways is a small Dublin startup which has problems with its current infrastructure, including downtime, vulnerabilities, and scalability issues. This project has to do with proposing AWS as a solution to a better and safer infrastructure, with a diagram, a proof of concept, and explanations. This infrastructure has both higher availability and resilience.

This project essentially shows how AWS can help WinLocal by providing services such as automated scaling, access control and recovery. And with the proof of concept, components such as load balancing, database integration, networking, and computing were implemented.

Task 1: Detailed AWS Architecture Diagram

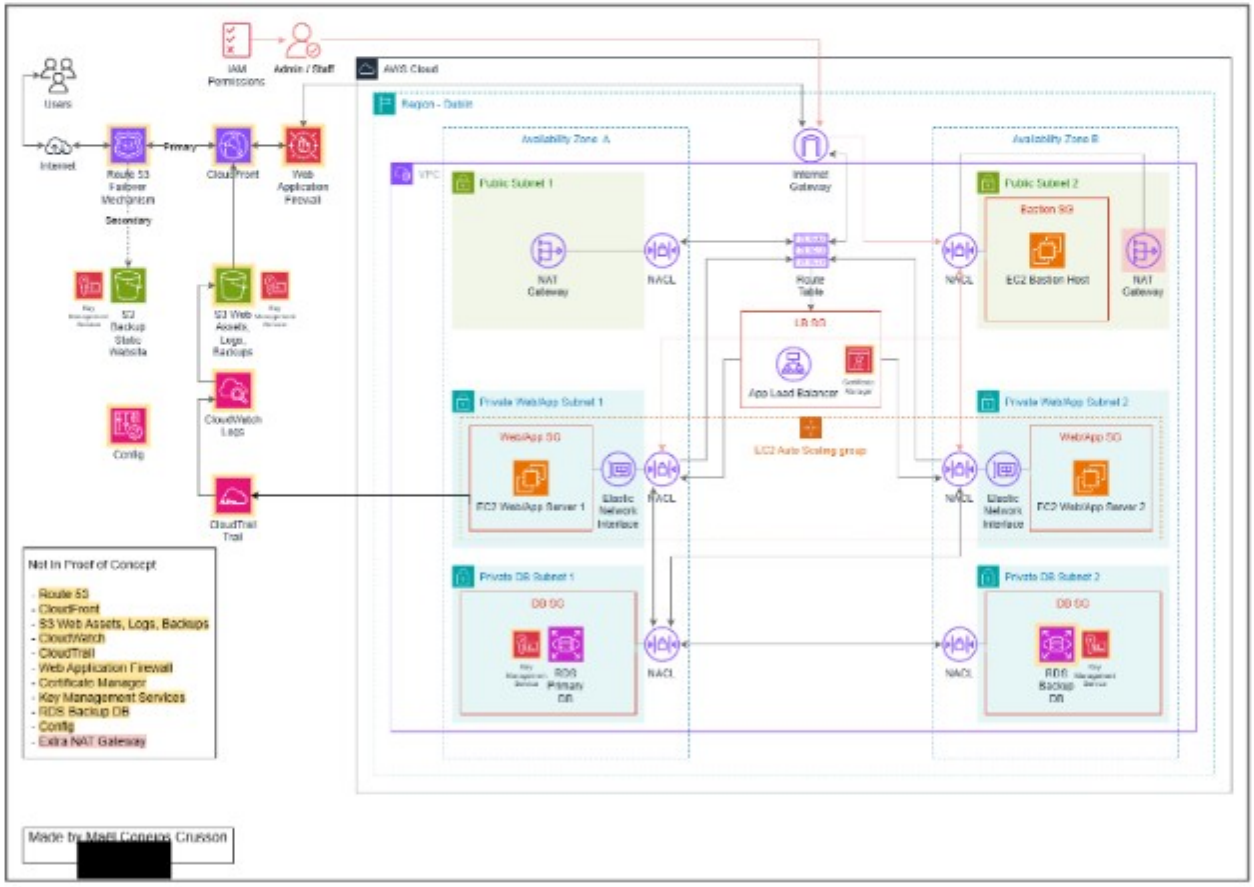


Figure 1: WinLocal Giveaways AWS Architecture Diagram

*** Explanations for Extra Elements**

For availability, I added Route 53 since it manages the automatic DNS-based failover requested by WinLocal. I also added CloudFront as it caches the content at edge locations closer to users which reduces latency as WinLocal will expand beyond Dublin, also adding protection against Denial-of-Service attacks. I finally added an extra NAT gateway in the second public subnet as if the first availability zone becomes unavailable, the second one will be available so that the private subnets will still have internet access. However, having the additional NAT is not recommended (as labelled in red) for now for WinLocal as having only one is more cost-conscious.

For security and governance, I added a web application firewall to protect the web application from web exploits (explained later). I also added a certificate manager for the app load balancer as it ensures traffic is encrypted in transit, supporting the secure by design requirement. I added key management service for the databases and s3 buckets as it creates cryptographic keys to encrypt data at rest. And I finally added AWS config as it provides an inventory of the AWS resources which allows the staff to see their environment & detected unauthorised changes.

Finally for monitoring and data management I added CloudWatch & CloudTrail as CloudWatch monitors performance and health while CloudTrail logs. I added as well the S3 web assets, logs, and backups as per WinLocal's request and a second RDS database in case the primary one fails (also explained later).

Task 2: Proof of Concept – Implementation & Explanation

Networking Design

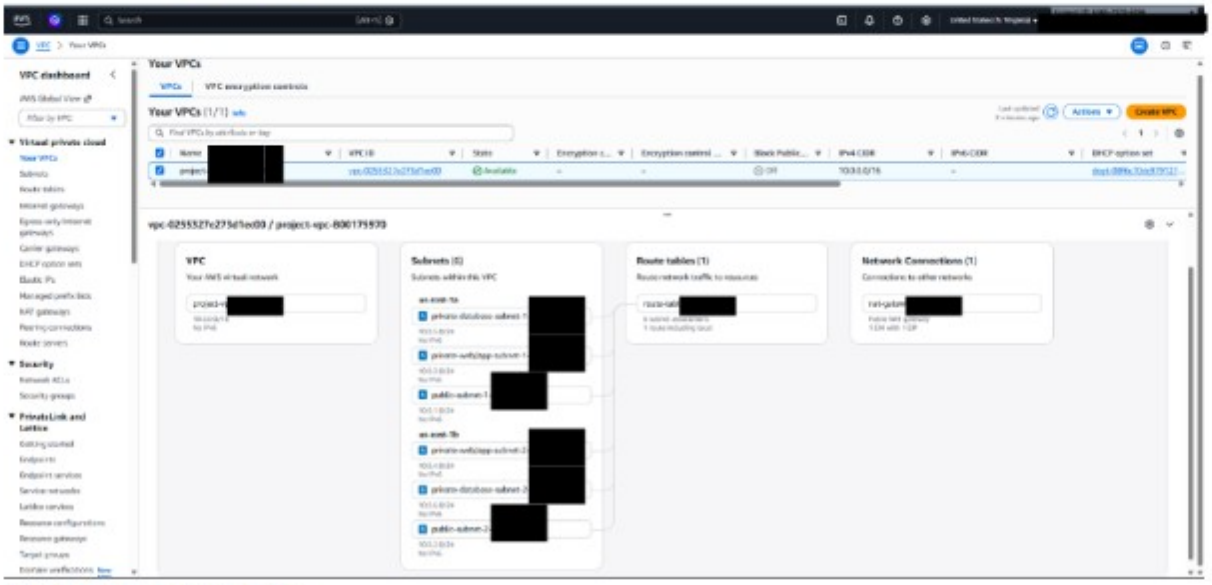


Figure 2: VPC Resource Map

For my logical design (Fig. 2) for WinLocal, I separated the resources across two availability zones and 3 subnets each. “Using multiple availability zones improves workload availability, reducing the risk of downtime caused by a single zone failure” (Tozzi, 2023). As Tozzi explained, I used two different availability zones because this layout provides high availability, fault tolerance, and more resilience against data centre failures. I then used 3 different subnets per AZ: one is public, where the NAT Gateway or the EC2 Bastion Host reside, one is private, exclusively for the EC2 Web/App Servers, and the last is also private, exclusively for the RDS databases (only the primary database is configured in the proof of concept). I then included a route table so that network traffic knows where to go.

Bastion Host

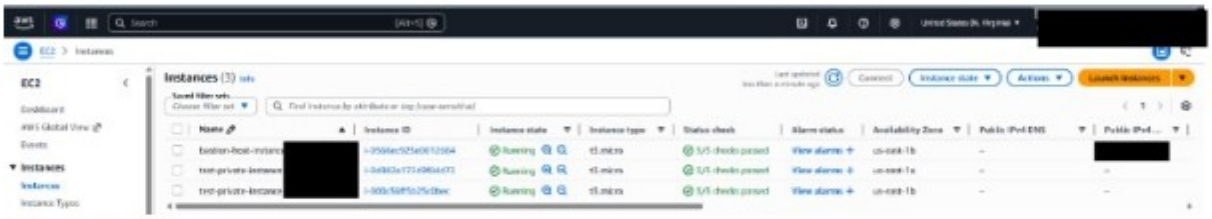


Figure 3: EC2 Instances

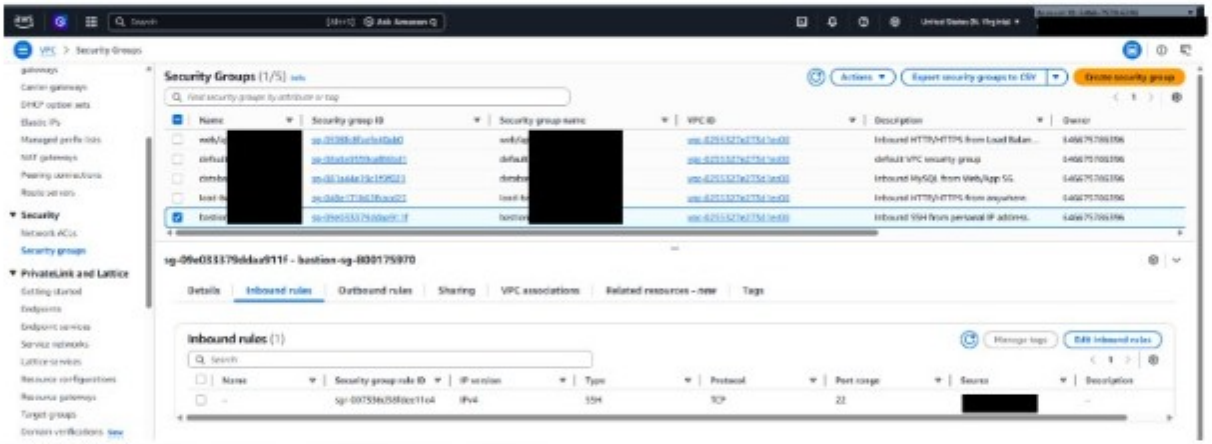


Figure 16: Security Groups - Bastion SG Inbound Rules



Figure 4: SSH to private Web/App Server via Bastion Host

A Bastion Host is needed as it's the bridge between the admin/staff and the private EC2 servers (instances in Fig. 3). As Rico stated, "a Bastion host is a secure access point, allowing administrators to connect to private instances via SSH or RDP" (Rico, 2024). The staff can SSH into the Bastion (security group in Fig.16), which is in a public subnet so they may access it via its public IP address since it is directly connected to the internet. The user then, from the Bastion Host, SSHed again into the server using the private IP address of the private EC2 instance (Fig. 4). So, a Bastion Host provides controlled remote access to internal infrastructure, which protects against unauthorized access. Additionally, regarding costs, using t3.micro as I did (Fig. 3) for the EC2 instances saves money as they are part of the AWS Free Tier, which means for the first 12 months they could run the proof of concept for almost €0. And also, using Amazon Linux is better for both EC2 instances rather than Windows as Windows would require licensing fees. Linux doesn't, therefore saving additional unnecessary costs.

Amazon Machine Image (AMI)

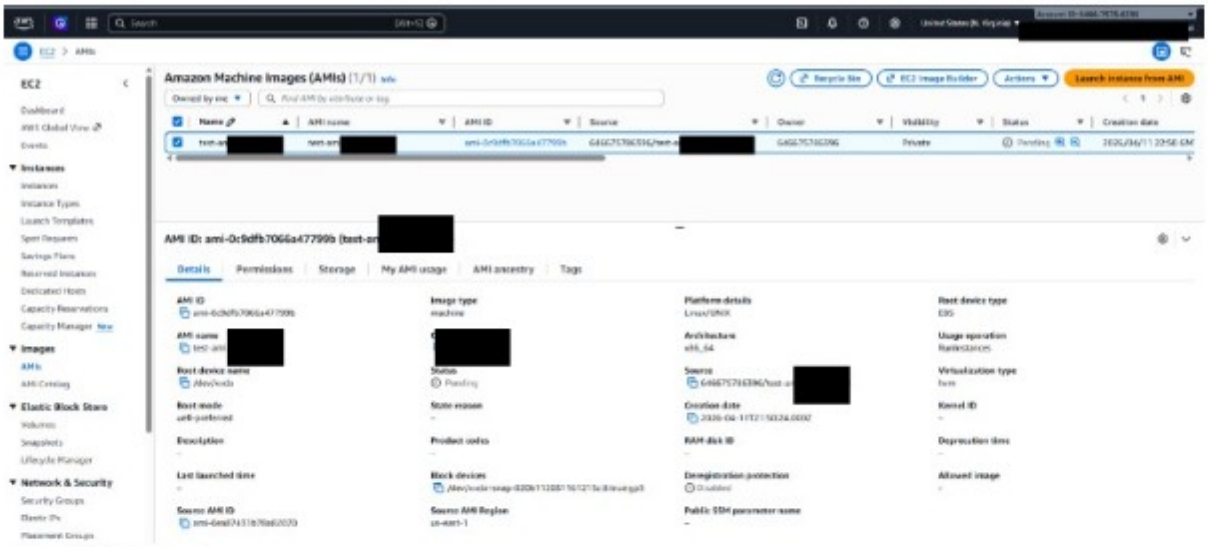


Figure 5: AMI Details

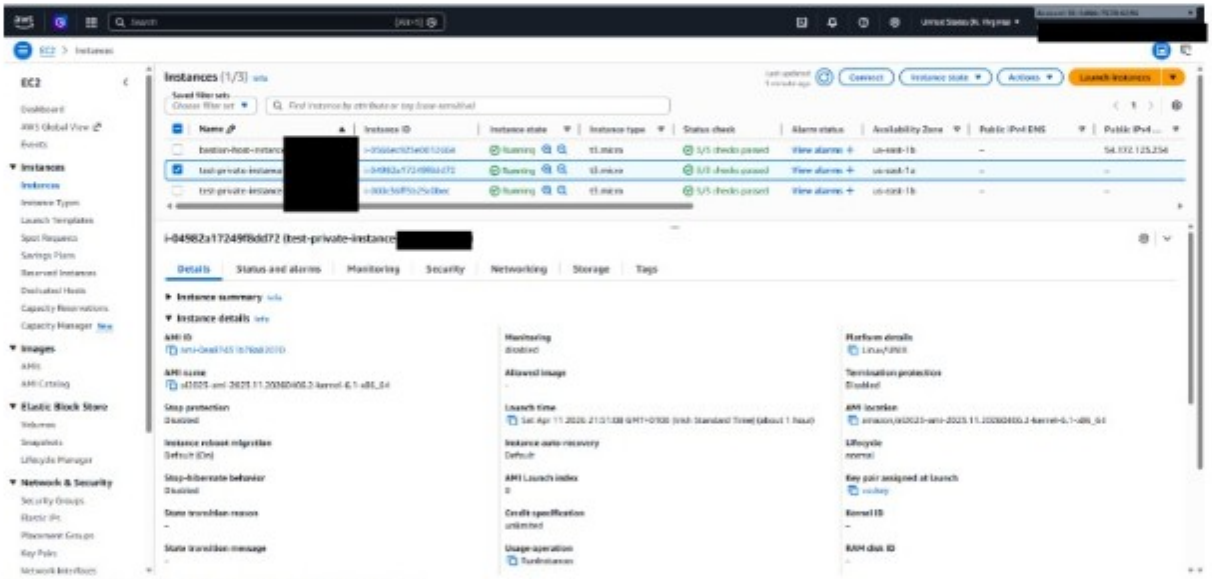


Figure 6: Web/App Server 1's AMI Details

Amazon Machine Images are useful as they provide a template which is pre-configured which can be used to launch other instances. For example, the current AMI (Fig. 5) takes from the private instance 1 (Fig. 6). They contain the operating system, the application server, and the software to run an application. This saves both time and effort.

Single-AZ Amazon RDS Deployment

```
[ec2-user@ip-10-0-3-204 ~]$ mysql -h test-databas[redacted]dujz80hfnmt.us-east-1.rds.amazonaws.com -P 3306 -u admin -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MySQL connection id is 43
Server version: 8.4.8 Source distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]> []
```

Figure 7: Connection to RDS from Web/App Server 1 via Bastion Host

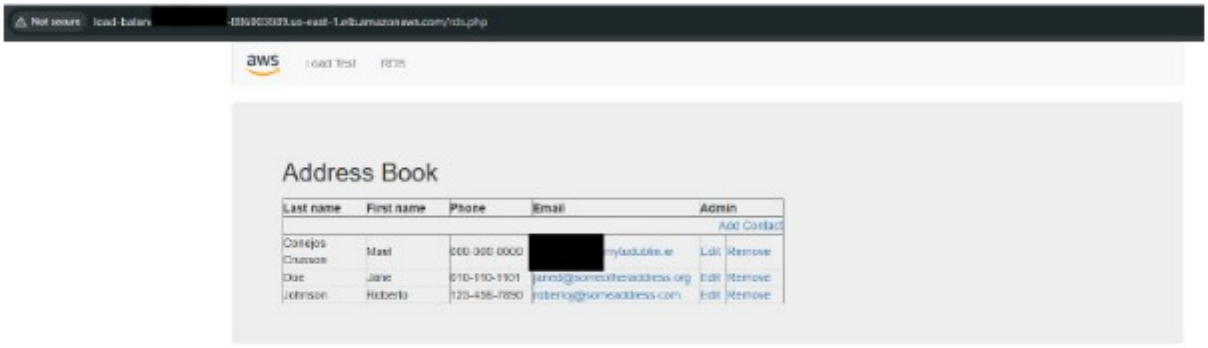


Figure 8: Database Contents from Load Balancer DNS Link

Here we can connect a database with the instances directly: the database is accessible from the private instances through the Bastion Host (Fig. 7), and it is accessible through the load balancer's DNS address (Fig. 8). There are two limitations with this current deployment in the proof of concept. The first one being the fact that it's a single-AZ RDS deployment, meaning the databases span only across one availability zone. This is bad as if there were to be a hardware-related or environmental-related problem regarding the AZ that renders the servers to be out of order, then nothing can access the databases associated with that AZ. The solution to this is to have multiple databases across multiple AZs (known as a multi-AZ RDS deployment). The second limitation is that this deployment only has one database, in case of a failure with the current database, there is no backup to support the instances related to it. The solution to this is to have a backup database that copies all of the information of the primary one, and acts as a failsafe in case the primary database fails.

Auto Scaling Configuration

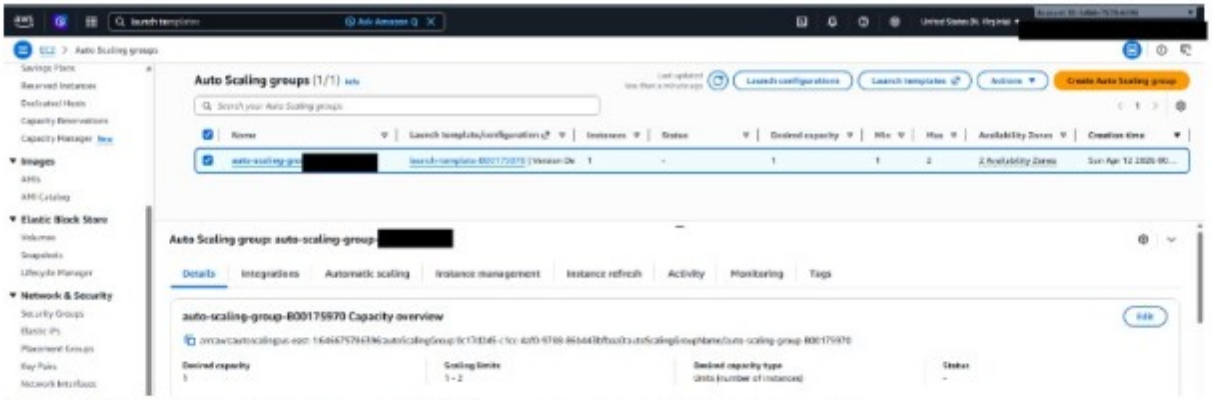


Figure 9: Auto Scaling - Desired capacity, Minimum capacity, and Maximum capacity

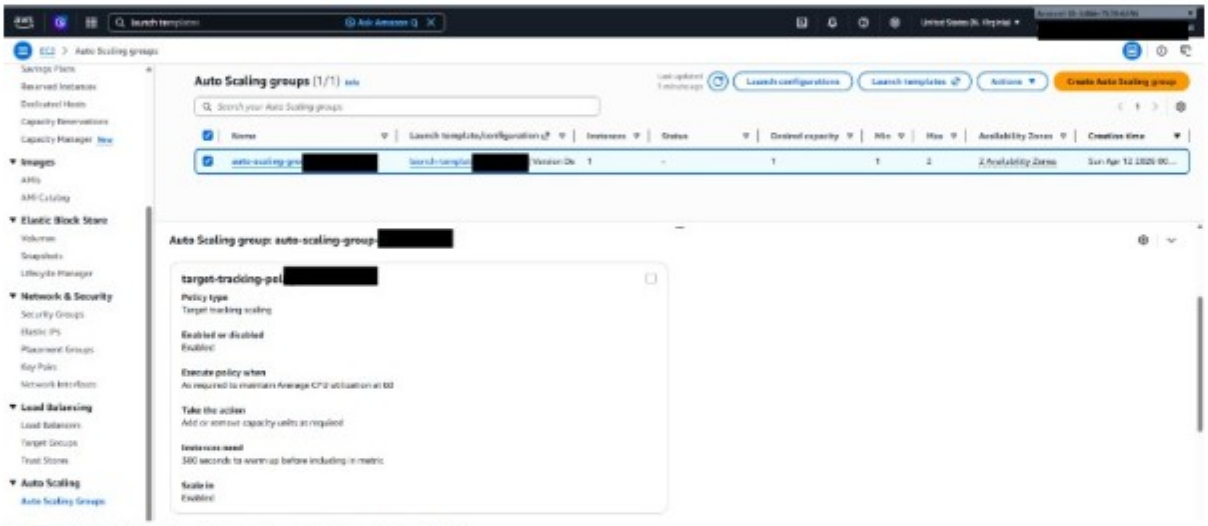


Figure 10: Auto Scaling – Target Tracking Policy

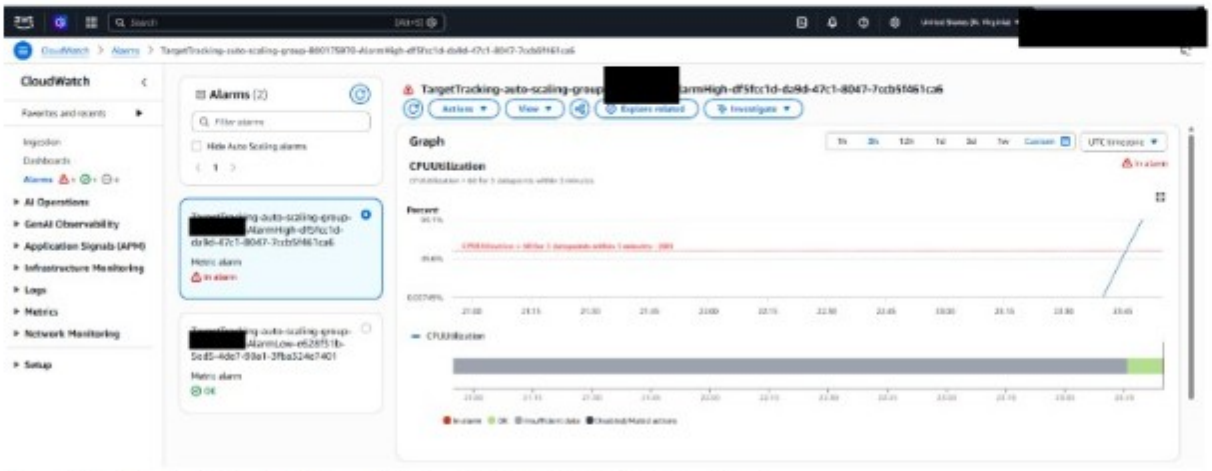


Figure 11: Auto Scaling - High Alarm Triggered (EC2 > 60% CPU utilisation)

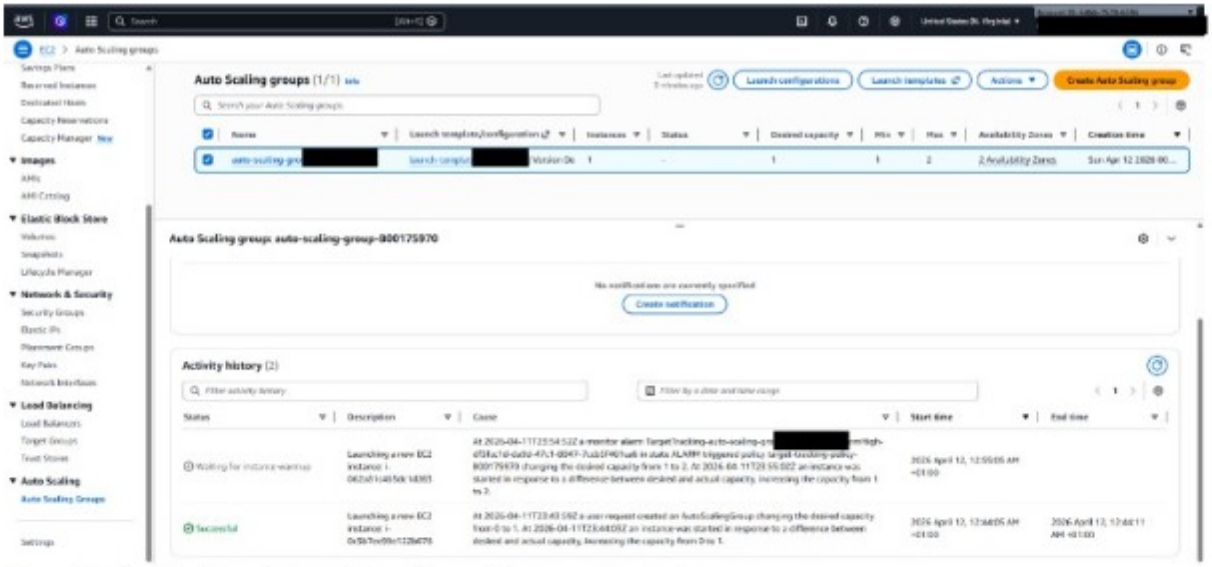


Figure 12: Auto Scaling - Activity History (Second Instance Created)

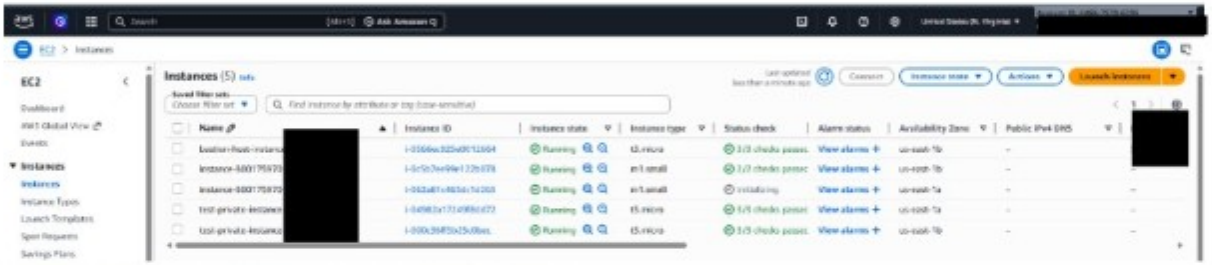


Figure 13: Auto Scaling - Two Extra Instances (The Max)

Auto Scaling happens with auto scaling groups that subnets are attached to. Each auto scaling group has parameters that dictate when additional instances must be created. As Jackson & Goodwin (2023) stated, auto scaling ensures that applications continuously have the resources they need to remain consistently available and meet performance goals, as well as promoting the efficient use of cloud resources and minimising the cost. In this context, WinLocal has requested that the scaling must have at least one instance, and a maximum of two instances (Fig. 9). The target is to keep the average CPU utilisation at 60% (Fig. 10), and if any instance exceeds that then an alarm triggers (Fig. 11) and the auto scaling adds a second instance in the pre-selected subnets (which in this case are the two private Web/App subnets) (Fig.12-13).

Load Balancer Configuration

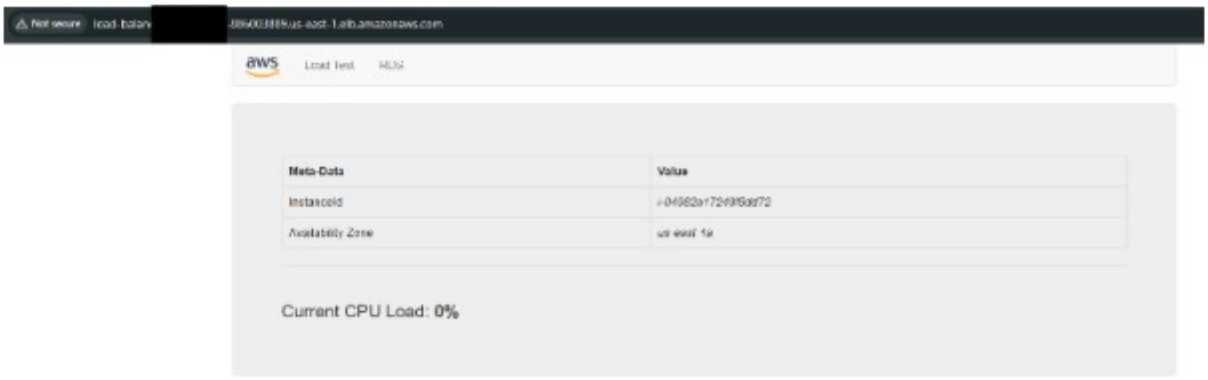


Figure 14: Web/App Server 1 displayed on Load Balancer DNS Link (only one instance appears at a time, after a page refresh)

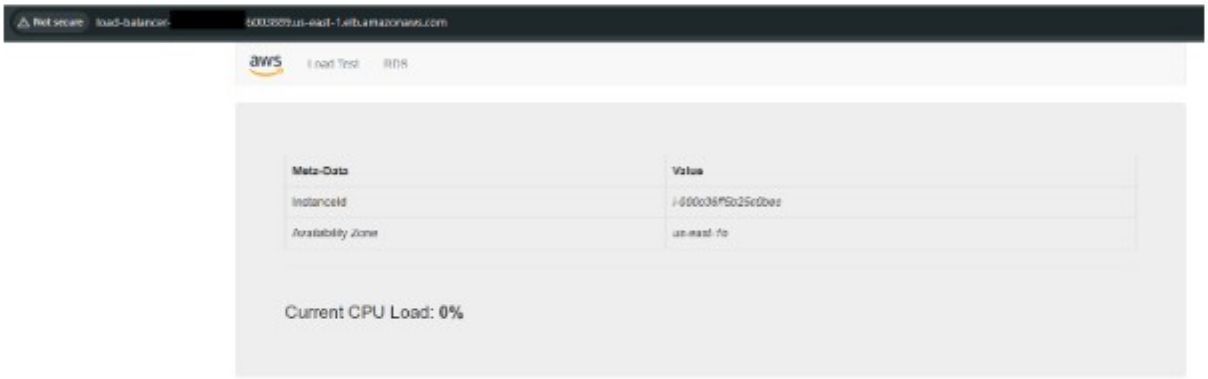


Figure 15: Web/App Server 2 displayed on Load Balancer DNS Link (only one instance appears at a time, after a page refresh)

Load Balancers distribute traffic across multiple servers, and this creates high availability as well as increased performance. It essentially prevents any single server from becoming overwhelmed, which can reduce latency and avoid crashes. Or as Cooper (2025) stated, it's a software sitting in a network that intelligently diverts network traffic among the available servers. So, if a server fails or is compromised, it will reroute traffic away from it to another.

Security Implementation

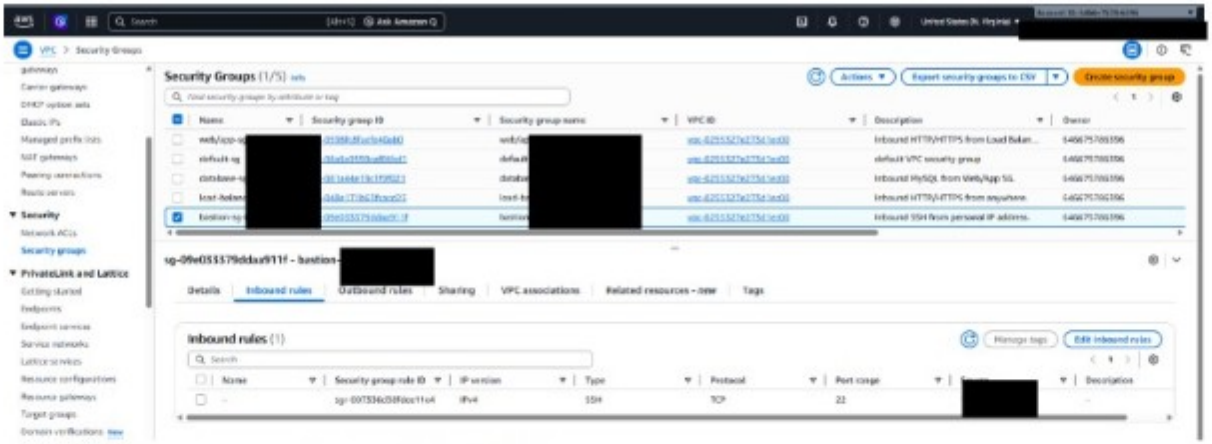


Figure 16: Security Groups - Bastion SG Inbound Rules

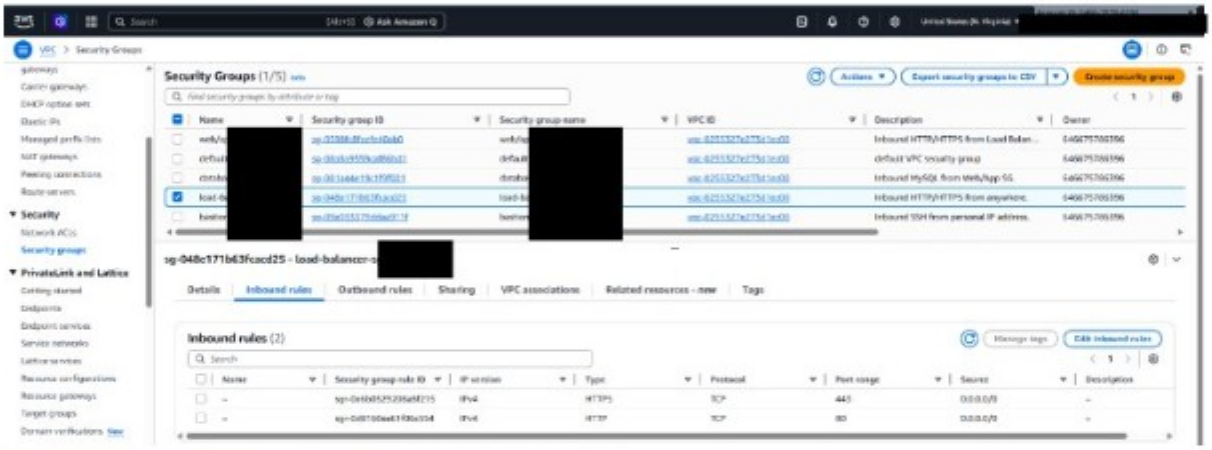


Figure 17: Security Groups - Load Balancer SG Inbound Rules

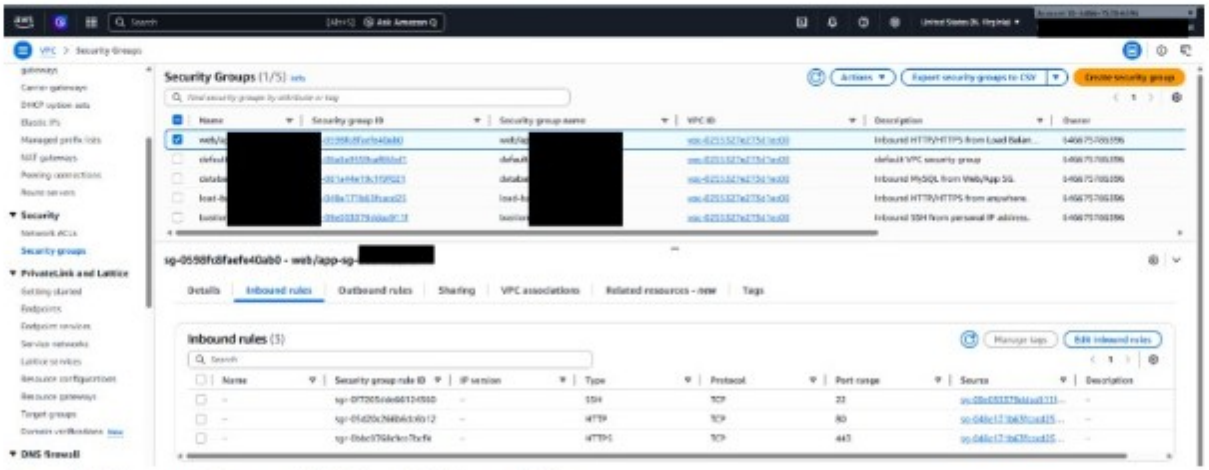


Figure 18: Security Groups - Web/App SG Inbound Rules

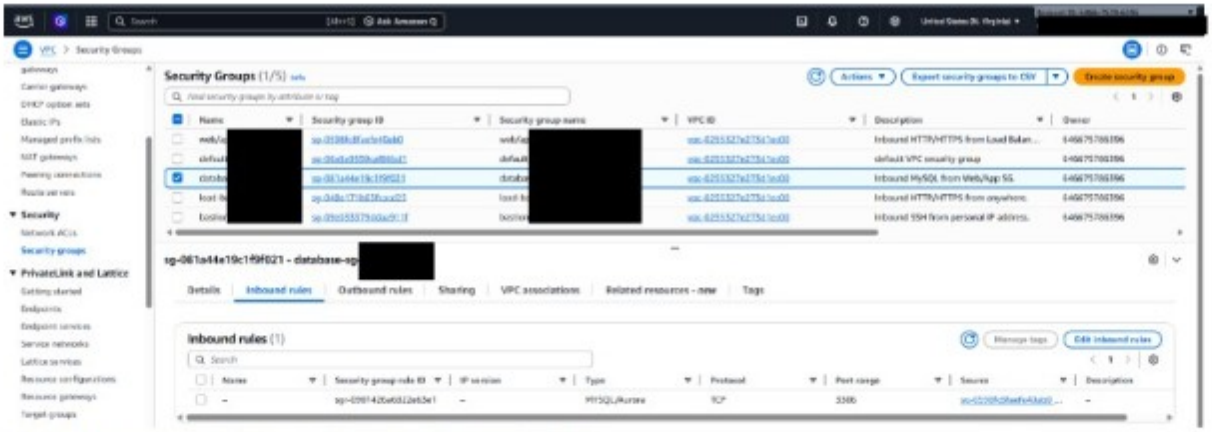


Figure 19: Security Groups - Database SG Inbound Rules

This system is secure as each section is isolated and only accessible from certain points. The security groups act as stateful firewalls at the instance level, which control inbound and outbound traffic. For the databases, their security group (Fig. 19) only allows inbound MySQL traffic from the web/app security group, which the web/app servers are attached to. This allows for zero access from unwanted places. For the web/app servers, their security group (Fig.18) allows inbound HTTP and HTTPS from the load balancer's security group only, preventing the private servers from being directly connected to the internet without passing through the load balancer. The web/app security group also allows inbound SSH from the bastion host, so that admins, developers, or staff may access the servers directly from their machine. The load balancer's security group (Fig. 17) allows inbound HTTP and HTTPS from anywhere, so that it is easily accessible for testing and development. However, it isn't the most secure, unless they are using HTTPS which encrypts the traffic. The final security group is the bastion host's' (Fig. 16), which allows inbound SSH from my current personal address only (or alternatively, the admin's IP address or a group associated with developers & staff only) to prevent public access to the servers.

Static Backup Website



Figure 20: S3 Bucket Policy

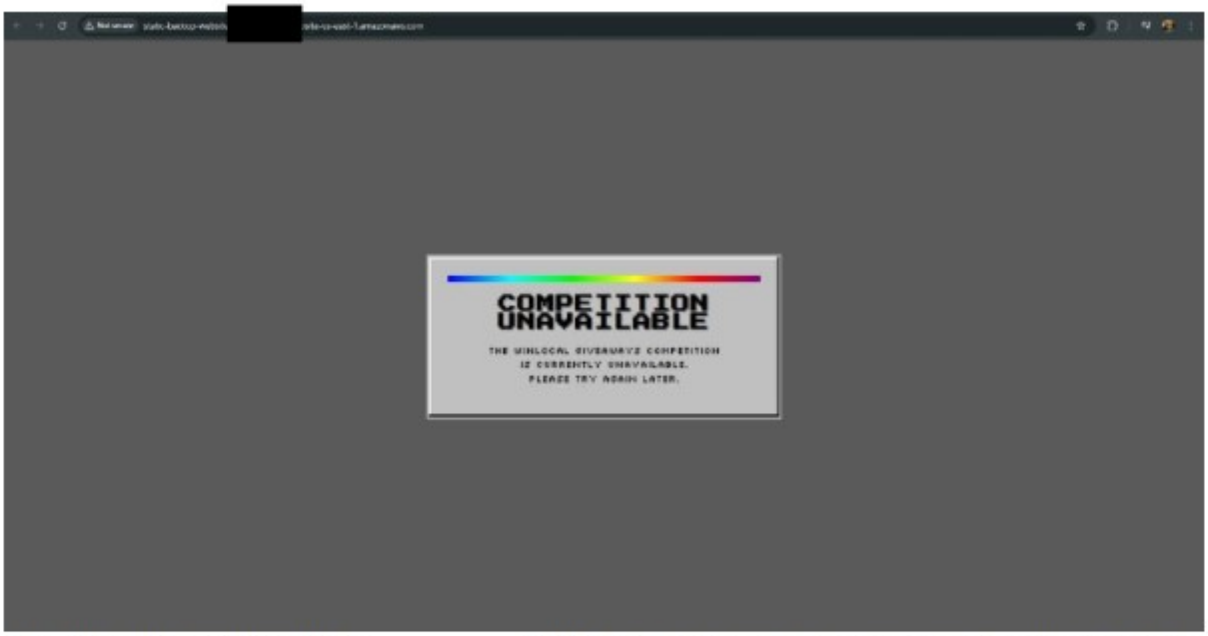


Figure 21: S3 Bucket Static Website **HTML & CSS code taken from a previous personal project of mine*

Firstly, using an S3 bucket rather than an EC2 instance for WinLocal's backup website is much more cost-efficient, as having an S3 bucket costs far less than a standby EC2 instance. The static backup website is useful in the event that the servers running the application are down, it has been hacked, or there is simply a problem with the connection. Or as Forrest (2022) puts it: "Simply put, a website backup is the single greatest insurance plan any website owner can invest in to prevent against website files loss". Having a static website in case of any unwanted event acts as a failsafe, redirecting the user to a simple page indicating, in this context, that the current competition is not available if trying to access an inaccessible WinLocal website.

Task 3: Justification of Additional AWS Services

Route 53

A service which was present in the diagram but not in the proof of concept was Route 53. The S3 bucket static website was made, but the failover mechanism to direct the user to that website wasn't implemented. GeeksforGeeks (2026) stated that Route 53 is overall a highly available and scalable domain name system web service for routing end users to internet applications.

Its purpose in this context is to provide a failover mechanism in case the primary application fails, redirecting the user to a static backup website. WinLocal needs it as their current setups lacks the failover mechanism, which leads to downtime during outages (for example, due to ransomware attacks). The benefit of using Route 53 is that it ensures high

availability, even if the static website (as of now) only indicates that the competition is not available. Route 53 would best be implemented with health checks and failover routing policies.

Web Application Firewall

Another service which is not present in the PoC is the web application firewall. As cloudchipr (2025) claims, WAF does several tasks which include filtering, monitoring, and blocking malicious HTTP and HTTPS traffic between a web application and the internet. It essentially protects servers from application-layer attacks (such as SQL injection or bots).

Its purpose in this context is to the web applications from common exploits. WinLocal needs this as their recent ransomware attack exploited vulnerabilities in their web application. The benefits of using WAF (as mentioned) are the blocking of malicious traffic as well as reducing the attack surface. The WAF should be placed before the Load Balancer (or attached to it).

Task 4: Sustainability – AWS Well-Architected Framework

The sustainability pillar is about having the least environmental impact while being as efficient as possible. In my proof of concept, I used auto scaling which scales the resources based on demand, reducing time idle and terminating unused resources, which saves both cost and energy. And I also implemented separate subnets which improve network efficiency.

My sustainability and budget advice for WinLocal to improve in the future is to firstly use spot instances for some workloads such as development or testing as they consume less power and are cheaper than alternatives. I would also recommend them to use reserved instances which both reduce costs and environmental impact, as well as using graviton-based instances for their servers as it uses ARM-based processors (like Snapdragon and Apple's M chips) which are more energy efficient. They could also have auto-shutdown schedules for non-production environments, saving energy. I would advise them as well to potentially implement a multi-AZ RDS deployment (as in my diagram). While having a standby database might consume more energy, it does reduce the need for over-provisioning. I would finally advise WinLocal to enable AWS Cost Explorer to monitor usage and waste.

Conclusion

In this project I designed and implemented an AWS solution for WinLocal Giveaways, helping them with the challenges they currently face (security, scalability, costs, and availability). Using services such as a virtual private network, a relational database service, auto scaling, and security groups. This solution has high availability, fault tolerance, and good security.

References

1. Tozzi, C. (2023). *Should You Use Multiple Availability Zones?* [online] Data Center Knowledge. Available at: https://www.datacenterknowledge.com/uptime/should-you-use-multiple-availability-zones-in-the-cloud- [Accessed 10 Apr. 2026].
2. Rico, N. (2024). *Understanding the Importance of Bastion Hosts in AWS*. [online] Tutorials Dojo. Available at: https://tutorialsdojo.com/understanding-the-importance-of-bastion-hosts-in-aws/ [Accessed 10 Apr. 2026].
3. Jackson, G. & Goodwin, M. (2023). *Auto scaling*. [online] IBM. Available at: https://www.ibm.com/think/topics/autoscaling [Accessed 10 Apr. 2026].
4. Cooper, K. (2025). *What's the point of load balancing?* [online] Loadbalancer. Available at: https://www.loadbalancer.org/blog/loadbalancing-101/ [Accessed 10 Apr. 2026].
5. Forrest, J. (2022). *5 reasons why it's important to perform a website backup*. [online] GoDaddy Resources. Available at: https://www.godaddy.com/resources/ae/skills/5-reasons-why-its-important-to-perform-a-website-backup [Accessed 10 Apr. 2026].
6. GeeksforGeeks (2026). *Introduction to Amazon Route53*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/cloud-computing/introduction-to-amazon-route53/ [Accessed 10 Apr. 2026].
7. cloudchipr (2025). *AWS WAF: Understanding Amazon's Web Application Firewall*. [online] cloudchipr. Available at: https://cloudchipr.com/blog/aws-waf [Accessed 10 Apr. 2026].